CSE144 Final Project Report
By: Ethan Phan

Team Contributions: All work was individual

Used a Jupyter notebook similar to previous homework assignments.

Initial approach:
For my initial approach, I wanted to keep a similar format to the previous assignment as I really liked the pipeline structure that the previous assignment had regarding using a jupyter notebook and I felt like I could distinguish how I wanted to train my model. I kept the same approach in saving the best trained epoch to the test loop. I started using RESNET-50 as a baseline since it was the easiest to augment and we learned about RESNET the most in class.

- Batch size = 32 to slowly train to prevent overfitting.
- RESNET has millions of parameters so starting small is the most ideal
- Cross entropy loss for the criterion
- Implemented AdamW and learning rate scheduler to decay the learning rate of each parameter group by gamma every step_size epochs.

Freezing RESNET layer for first epochs runs, then unfreezing and training with smaller learning rate. Trying to preserve pretrained features

Initial learning rate and weight decay:

Learning rate: 0.001

Weight decay: 0.0001

Main pipeline

Using a Jupyter notebook for easier accessibility and most straightforward. Validation and training functions are implemented similar to the previous assignment.

- Importing pytorch dependencies and initializing CPU and GPU environment

↓

- Loading dataset, resizing, transforms, and other data augmentation methods. Splitting dataset to test/train/val

↓

- Building CNN models using different models such as RESNET-50, EfficientNet-B0, and vit_b_16 and freezing the initial layers for all models.

↓

- Train function with optim hyper parameters such as cross entropy loss, AdamW, and reducedLRonPlateau.

↓

- Validation function that evaluates model, computes, and returns average loss and accuracy. Loops through batches without computing gradients.

↓

- Training loop that implements both the training and validation function. Also implements hyperparameters such as entropy loss, AdamW, and reducedLRonPlateau. Loop also uses early stopping with patience. This is the main loop that runs and the test and validation sets. Output prints Epoch, LR, Training Loss, Training Acc, Val Loss, and Val accuracy. Epoch with best validation accuracy was saved just like in homework 2

↓

- Testing loop that takes the highest saved epoch from training and runs the test set using model.eval() which was either EfficientNet or VIT. A CSV file is created from these predictions and sent to Kaggle.

Software Environment:

All of my testing was done with CPU. Initially, this was no problem with running as all the epochs and training were <40 minutes for 15-20 epochs. As I progressed with switching models and different hyperparameters. Training took much longer as 15 epochs on ViT can take around 3 hours or more. This was why a lot of my validation results were not as high due to how long my models took to run. My laptop also does not offer NVIDIA cudas.

```
torch: 2.9.0+cpu
device: cpu
Path to competition files: C:\Users\ethan\.cache\kagglehu
C:\Users\ethan\.cache\kagglehub\competitions\ucsc-cse-144
['sample_submission.csv', 'test', 'train']
```

Dataset and augmentation:

train/val/test: 863 216 1036

One of the main concerns I had with this approach was how little samples the validation set contained as other students had discussed about not splitting the training set into test/val. I felt like splitting it was better for data generalization.

The test set includes 1036 samples. I had many issues with labeling as my initial implementation had the labels hardcoded in instead of sampling random labels from tensor. A lot of the labels were not sampled randomly and I had created a separate class dataset for both the training and testing sets.

I also switched from using randomsplit in torch.generator when splitting the data to sklearn's train_test_split for better data generalization since the dataset was so small.

```
class CompetitionTrainDataset(Dataset):
    def __init__(self, root, transform=None):
        self.transform = transform
        self.samples = []

        for class_name in sorted(os.listdir(root), key=lambda x: int(x)):
            class_dir = os.path.join(root, class_name)
            if not os.path.isdir(class_dir):
                continue

            label = int(class_name)

            for fname in sorted(os.listdir(class_dir)):
                if fname.endswith(".jpg"):
                    self.samples.append((os.path.join(class_dir, fname),label))

    def __len__(self):
        return len(self.samples)

    def __getitem__(self, idx):
        img_path, label = self.samples[idx]

        image = Image.open(img_path).convert("RGB")
```

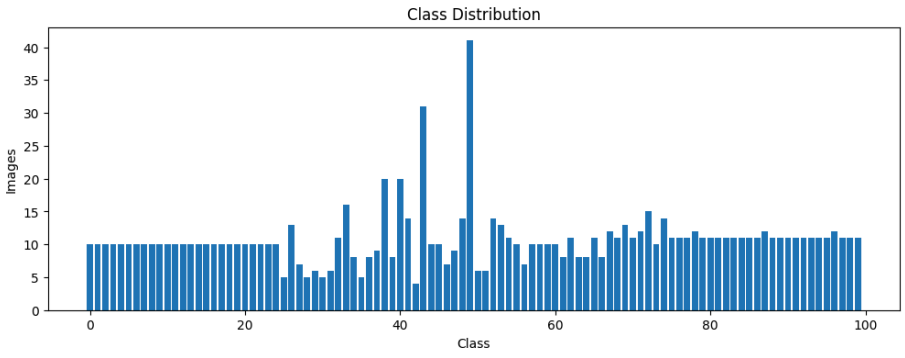
Using Sklearn and subset for data splitting.

```
labels = [label for _, label in full_train.samples]
indices = np.arange(len(full_train))
train_idx, val_idx = train_test_split(
    indices,
    test_size=0.2,
    random_state=42,
    stratify=labels
)

train_set = Subset(full_train, train_idx)
val_set = Subset(full_train, val_idx)
```

There were also issues with the split of training samples within classes. For example some classes may have an even 10 sample while others may have a smaller sample size such as 5 or 6.

```
Number of training images: 1079
train/val/test: 863 216 1036
Batch image shape: torch.Size([32, 3, 224, 224])
Batch labels shape: torch.Size([32])
Sample labels in batch: tensor([64, 49, 72, 52, 58, 74, 38, 67,  5, 12, 12, 21, 91, 94, 32,  5,  1, 38,
        94, 14])
Number of unique classes: 100
Minimum samples per class: 4
Maximum samples per class: 41
```

Uneven class distribution. In samples to classes. Many of the classes had uneven samples ranging from 4-40



Messing around with different augmentations such as RandomHorizontalFlip, VerticalFlip, and ColorJitter. All images were set to 224x224 Some of the augmentation methods such as AutoAugment seemed to lower the validation score.

```python
train_tf = transforms.Compose([
    transforms.Resize((224, 224), interpolation=InterpolationMode.BICUBIC),
    transforms.RandomHorizontalFlip(),  # Data augmentation
    transforms.RandomVerticalFlip(),  # Data augmentation
    #transforms.RandomRotation(10),  # Data augmentation
    transforms.RandomResizedCrop(224, scale=(0.8, 1.0), interpolation=InterpolationMode.BICUBIC),  # Data augmentation
    #transforms.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2, hue=0.1),  # Data augmentation
    transforms.AutoAugment(transforms.autoaugment.AutoAugmentPolicy.IMAGENET),  # Data augmentation
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],std=[0.229,  0.224, 0.225])])


test_tf = transforms.Compose([
    transforms.Resize((224, 224), interpolation=InterpolationMode.BICUBIC),  # ResNet50 expects 224x224 input
    transforms.CenterCrop(224),  # Ensure consistent cropping
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])])
```

Models:

Weights for all models: ResNet50_weights, EfficientNet_B0_Weights, Vit_B_16_Weights

Hyperparameters: Cross entropy loss with label smoothing = 0.1, AdamW = learning rate at 0.0001 and weight decay at 0.00001, schedular with reduceLRonPlateau.

Epochs: Started small around 8 epochs. As I decreased the learning rate, the number of epochs increased. The most epochs ran was 23 to save time. I averaged around 10-15 epochs for each run.

```python
criterion = nn.CrossEntropyLoss(label_smoothing=0.1)
optimizer = torch.optim.AdamW(filter(lambda p: p.requires_grad, model.parameters()), lr=1e-4, weight_decay=1e-5)

scheduler1 = torch.optim.lr_scheduler.StepLR(optimizer, step_size=5, gamma=0.5)
```

```python
scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode='max', factor=0.5, patience=3)

patience = 6
epochs_without_improvement = 0
```

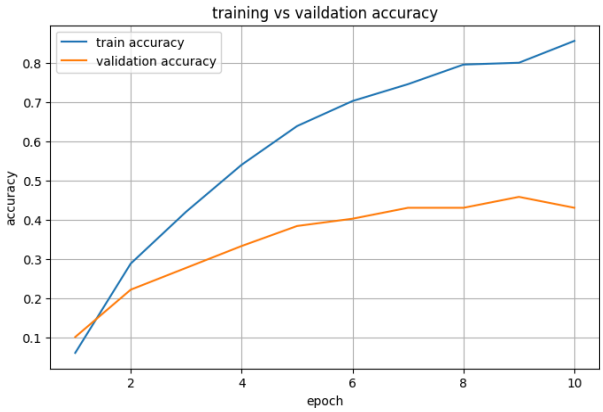
RESNET:

RESNET-50 was the baseline and simplest to implement initially because of how much it was discussed in class. The augmentation and layer concatenation was extremely easy to follow. It was simple as well to train the hyperparameters, learning rate, weight decay, etc. In total the model has 4 layers while each has 2 to 5 bottleneck layers. For input features I implemented a fully connected layer with linear transformations and output features set to 512. Input channels are floating around 3 to 64. I also added Relu as an activation function and applied dropout to

0.3 in the fully connected layer. In total there were around 20 million parameters but I implemented layer freezing in which then lowered the parameters to 11 million.

When running with RESNET, the baseline accuracy was particularly low as validation accuracy was around ~40-45% for 10 epochs. All layers were frozen as well during this first time running the model. After seeing other student's presentations and their implementations, I decided to make a switch. I did not realize that RESNET was one of the older neural networks.

```
lass KaggleResNet(nn.Module):
  def __init__(self, num_classes=100):
      super().__init__()
      self.model = resnet50(weights=ResNet50_Weights.IMAGENET1K_V2)

      for param in self.model.parameters():
          param.requires_grad = False

      in_features = self.model.fc.in_features
      self.model.fc = nn.Sequential(
          nn.Linear(in_features, 256),
          nn.ReLU(),
          nn.Dropout(0.5),
          nn.Linear(256, num_classes)
      )

  def forward(self, x):
      # Pass input through features, then classifier
      return self.model(x)
```



Switching to EfficentNet-B0:

EfficientNet was recommended to me by other students and the sequential classifier was much shorter to implement as I only added the dropout and Linear. I experimented with layer freezing and unfreezing the most with EfficinetNet. I started with training the head and then slowly unfreezing after each time running. For example I started when param = false. I then set self.model.features[-1:], lowering learning rate, and then running. Repeat until I get to [-4:].

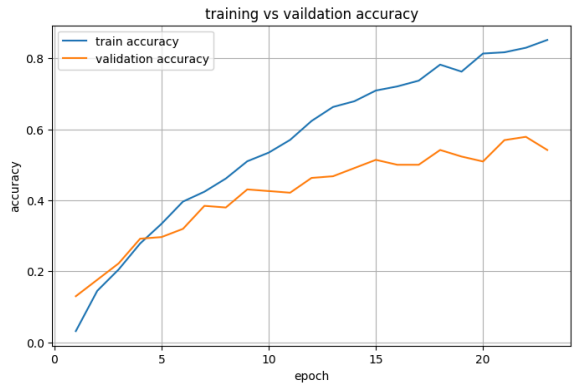
```python
class KaggleEfficientNet(nn.Module):
    def __init__(self, num_classes=100):
        super().__init__()
        self.model = efficientnet_b0(weights=EfficientNet_B0_Weights.IMAGENET1K_V1)

        for param in self.model.parameters():
            param.requires_grad = False

        for param in self.model.features[-3:].parameters():
            param.requires_grad = True

        in_features = self.model.classifier[1].in_features
        self.model.classifier = nn.Sequential(
            nn.Dropout(0.5),
            nn.Linear(in_features, num_classes)
        )

    def forward(self, x):
        return self.model(x)
```

Efficient net after 23 epochs: Best Val at 0.57.

This was when I had yet created the test loop and was also planning on switching to another model. If I ran this model in the test, my score most likely would have been higher on the test.

```
Epoch [1/23] | LR: 0.000100 | Train Loss: 4.5615 | Train Acc: 0.0313 | Val Loss: 4.4145 | Val Acc: 0.1296
Epoch [2/23] | LR: 0.000100 | Train Loss: 4.2916 | Train Acc: 0.1448 | Val Loss: 4.1488 | Val Acc: 0.1759
Epoch [3/23] | LR: 0.000100 | Train Loss: 3.9880 | Train Acc: 0.2051 | Val Loss: 3.8747 | Val Acc: 0.2222
Epoch [4/23] | LR: 0.000100 | Train Loss: 3.7181 | Train Acc: 0.2781 | Val Loss: 3.5985 | Val Acc: 0.2917
Epoch [5/23] | LR: 0.000100 | Train Loss: 3.4749 | Train Acc: 0.3337 | Val Loss: 3.3851 | Val Acc: 0.2963
Epoch [6/23] | LR: 0.000100 | Train Loss: 3.2304 | Train Acc: 0.3963 | Val Loss: 3.1619 | Val Acc: 0.3194
Epoch [7/23] | LR: 0.000100 | Train Loss: 3.0334 | Train Acc: 0.4241 | Val Loss: 3.0111 | Val Acc: 0.3843
Epoch [8/23] | LR: 0.000100 | Train Loss: 2.8634 | Train Acc: 0.4612 | Val Loss: 2.8608 | Val Acc: 0.3796
Epoch [9/23] | LR: 0.000100 | Train Loss: 2.6850 | Train Acc: 0.5098 | Val Loss: 2.7652 | Val Acc: 0.4306
Epoch [10/23] | LR: 0.000100 | Train Loss: 2.5800 | Train Acc: 0.5342 | Val Loss: 2.6560 | Val Acc: 0.4259
Epoch [11/23] | LR: 0.000100 | Train Loss: 2.4475 | Train Acc: 0.5701 | Val Loss: 2.5719 | Val Acc: 0.4213
Epoch [12/23] | LR: 0.000100 | Train Loss: 2.3330 | Train Acc: 0.6234 | Val Loss: 2.5219 | Val Acc: 0.4630
Epoch [13/23] | LR: 0.000100 | Train Loss: 2.2262 | Train Acc: 0.6628 | Val Loss: 2.4414 | Val Acc: 0.4676
Epoch [14/23] | LR: 0.000100 | Train Loss: 2.1112 | Train Acc: 0.6790 | Val Loss: 2.3471 | Val Acc: 0.4907
Epoch [15/23] | LR: 0.000100 | Train Loss: 2.0104 | Train Acc: 0.7092 | Val Loss: 2.3169 | Val Acc: 0.5139
Epoch [16/23] | LR: 0.000100 | Train Loss: 1.9600 | Train Acc: 0.7207 | Val Loss: 2.2741 | Val Acc: 0.5000
Epoch [17/23] | LR: 0.000100 | Train Loss: 1.8686 | Train Acc: 0.7370 | Val Loss: 2.2419 | Val Acc: 0.5000
Epoch [18/23] | LR: 0.000100 | Train Loss: 1.7818 | Train Acc: 0.7822 | Val Loss: 2.1958 | Val Acc: 0.5417
Epoch [19/23] | LR: 0.000100 | Train Loss: 1.7459 | Train Acc: 0.7625 | Val Loss: 2.1747 | Val Acc: 0.5231
Epoch [20/23] | LR: 0.000100 | Train Loss: 1.6507 | Train Acc: 0.8134 | Val Loss: 2.1602 | Val Acc: 0.5093
Epoch [21/23] | LR: 0.000100 | Train Loss: 1.6147 | Train Acc: 0.8169 | Val Loss: 2.1061 | Val Acc: 0.5694
Epoch [22/23] | LR: 0.000100 | Train Loss: 1.5427 | Train Acc: 0.8297 | Val Loss: 2.0584 | Val Acc: 0.5787
Epoch [23/23] | LR: 0.000100 | Train Loss: 1.4930 | Train Acc: 0.8517 | Val Loss: 2.0965 | Val Acc: 0.5417
Best val acc: 0.5787037037037037 at epoch 22
```

A little overfitting



Switching to ViT:
VIT backbone structure: 222,564 parameters. VIT model caused lots of initial underfitting. Highest initial validation accuracy was 0.15. My implementation of freezing and unfreezing layers was similar to that of EfficientNet. I once again started when param = false. I then set self.model.features[-1:], lowering learning rate, and then running. Repeat until I get to [-4:].

The main issue with running ViT was that it was extremely slow. This may have been possibly due to an overworked CPU after training for an extended period.

VIT Model and weights: Unfrozen 2 layers.

```python
class VIT(nn.Module):
    def __init__(self, num_classes=100):
        super().__init__()
        self.model = vit_b_16(weights=ViT_B_16_Weights.IMAGENET1K_V1)

        for param in self.model.parameters():
            param.requires_grad = False

        for block in self.model.encoder.layers[-2:]:
            for param in block.parameters():
                param.requires_grad = True

        in_features = self.model.heads.head.in_features
        self.model.heads.head = nn.Sequential(
            nn.Linear(in_features, 256),
            nn.ReLU(),
            nn.Dropout(0.5),
            nn.Linear(256, num_classes)
        )

    def forward(self, x):
        return self.model(x)
```

VIT after 15 epochs: Best Val at 0.56
15 was the sweet spot. Anything over 20 was too long and became redundant for training.

```
Epoch [1/12] | LR: 0.000100 | Train Loss: 4.5457 | Train Acc: 0.0394 | Val Loss: 4.3642 | Val Acc: 0.0972
Epoch [2/12] | LR: 0.000100 | Train Loss: 4.2470 | Train Acc: 0.1159 | Val Loss: 4.0406 | Val Acc: 0.1620
Epoch [3/12] | LR: 0.000100 | Train Loss: 3.9211 | Train Acc: 0.1958 | Val Loss: 3.7453 | Val Acc: 0.2222
Epoch [4/12] | LR: 0.000100 | Train Loss: 3.6158 | Train Acc: 0.2503 | Val Loss: 3.4581 | Val Acc: 0.2824
Epoch [5/12] | LR: 0.000100 | Train Loss: 3.3274 | Train Acc: 0.3117 | Val Loss: 3.2564 | Val Acc: 0.3194
Epoch [6/12] | LR: 0.000100 | Train Loss: 3.0502 | Train Acc: 0.4056 | Val Loss: 3.0467 | Val Acc: 0.3565
Epoch [7/12] | LR: 0.000100 | Train Loss: 2.8366 | Train Acc: 0.4357 | Val Loss: 2.8328 | Val Acc: 0.4213
Epoch [8/12] | LR: 0.000100 | Train Loss: 2.6339 | Train Acc: 0.4890 | Val Loss: 2.7563 | Val Acc: 0.4259
Epoch [9/12] | LR: 0.000100 | Train Loss: 2.4239 | Train Acc: 0.5411 | Val Loss: 2.5864 | Val Acc: 0.4444
Epoch [10/12] | LR: 0.000100 | Train Loss: 2.2816 | Train Acc: 0.5944 | Val Loss: 2.4775 | Val Acc: 0.5324
Epoch [11/12] | LR: 0.000100 | Train Loss: 2.1255 | Train Acc: 0.6327 | Val Loss: 2.4174 | Val Acc: 0.4907
Epoch [12/12] | LR: 0.000100 | Train Loss: 1.9788 | Train Acc: 0.6964 | Val Loss: 2.2974 | Val Acc: 0.5324
Best val acc: 0.5324074074074074 at epoch 10
```

Vit initial frozen backbone. After unfreezing 2 layers, parameters were ~14,000,000

```
VIT(
  (model): VisionTransformer(
    (conv_proj): Conv2d(3, 768, kernel_size=(16, 16), stride=(16, 16))
    (encoder): Encoder(
      (dropout): Dropout(p=0.0, inplace=False)
      (layers): Sequential(
        (encoder_layer_0): EncoderBlock(
          (ln_1): LayerNorm((768,), eps=1e-06, elementwise_affine=True)
          (self_attention): MultiheadAttention(
            (out_proj): NonDynamicallyQuantizableLinear(in_features=768, out_features=768, bias=True)
          )
          (dropout): Dropout(p=0.0, inplace=False)
          (ln_2): LayerNorm((768,), eps=1e-06, elementwise_affine=True)
          (mlp): MLPBlock(
            (0): Linear(in_features=768, out_features=3072, bias=True)
            (1): GELU(approximate='none')
            (2): Dropout(p=0.0, inplace=False)
            (3): Linear(in_features=3072, out_features=768, bias=True)
            (4): Dropout(p=0.0, inplace=False)
          )
        )
        (encoder_layer_1): EncoderBlock(
          (ln_1): LayerNorm((768,), eps=1e-06, elementwise_affine=True)
          (self_attention): MultiheadAttention(
            (out_proj): NonDynamicallyQuantizableLinear(in_features=768, out_features=768, bias=True)
...
    )
  )
)
Total params: 222564
```

2 layers unfrozen:

```
VIT(
  (model): VisionTransformer(
    (conv_proj): Conv2d(3, 768, kernel_size=(16, 16), stride=(16, 16))
    (encoder): Encoder(
      (dropout): Dropout(p=0.0, inplace=False)
      (layers): Sequential(
        (encoder_layer_0): EncoderBlock(
          (ln_1): LayerNorm((768,), eps=1e-06, elementwise_affine=True)
          (self_attention): MultiheadAttention(
            (out_proj): NonDynamicallyQuantizableLinear(in_features=768, out_features=768, bias=True)
          )
          (dropout): Dropout(p=0.0, inplace=False)
          (ln_2): LayerNorm((768,), eps=1e-06, elementwise_affine=True)
          (mlp): MLPBlock(
            (0): Linear(in_features=768, out_features=3072, bias=True)
            (1): GELU(approximate='none')
            (2): Dropout(p=0.0, inplace=False)
            (3): Linear(in_features=3072, out_features=768, bias=True)
            (4): Dropout(p=0.0, inplace=False)
          )
        )
        (encoder_layer_1): EncoderBlock(
          (ln_1): LayerNorm((768,), eps=1e-06, elementwise_affine=True)
          (self_attention): MultiheadAttention(
            (out_proj): NonDynamicallyQuantizableLinear(in_features=768, out_features=768, bias=True)
...
    )
  )
)
Total params: 14398308
```

Final VIT training run at 20 epochs, LR = 0.001, Weight decay = 0.0001: Best Val at 0.699
There was some decline a little bit around epoch 11, but accuracy increased after 14.

```
Epoch [1/20] | LR: 0.000100 | Train Loss: 3.7171 | Train Acc: 0.2399 | Val Loss: 3.5434 | Val Acc: 0.2454
Epoch [2/20] | LR: 0.000100 | Train Loss: 3.3278 | Train Acc: 0.3511 | Val Loss: 3.2013 | Val Acc: 0.3565
Epoch [3/20] | LR: 0.000100 | Train Loss: 2.9906 | Train Acc: 0.4276 | Val Loss: 2.9601 | Val Acc: 0.4352
Epoch [4/20] | LR: 0.000100 | Train Loss: 2.7218 | Train Acc: 0.4936 | Val Loss: 2.7483 | Val Acc: 0.4583
Epoch [5/20] | LR: 0.000100 | Train Loss: 2.4416 | Train Acc: 0.5736 | Val Loss: 2.5625 | Val Acc: 0.4954
Epoch [6/20] | LR: 0.000100 | Train Loss: 2.2142 | Train Acc: 0.6454 | Val Loss: 2.4135 | Val Acc: 0.5324
Epoch [7/20] | LR: 0.000100 | Train Loss: 2.0451 | Train Acc: 0.6895 | Val Loss: 2.2864 | Val Acc: 0.5648
Epoch [8/20] | LR: 0.000100 | Train Loss: 1.8796 | Train Acc: 0.7277 | Val Loss: 2.1740 | Val Acc: 0.5787
Epoch [9/20] | LR: 0.000100 | Train Loss: 1.6928 | Train Acc: 0.7949 | Val Loss: 2.1492 | Val Acc: 0.5972
Epoch [10/20] | LR: 0.000100 | Train Loss: 1.5508 | Train Acc: 0.8459 | Val Loss: 2.0750 | Val Acc: 0.5926
Epoch [11/20] | LR: 0.000100 | Train Loss: 1.4454 | Train Acc: 0.8656 | Val Loss: 2.0700 | Val Acc: 0.5880
Epoch [12/20] | LR: 0.000100 | Train Loss: 1.3980 | Train Acc: 0.8899 | Val Loss: 2.0360 | Val Acc: 0.5694
Epoch [13/20] | LR: 0.000100 | Train Loss: 1.3008 | Train Acc: 0.9061 | Val Loss: 2.0016 | Val Acc: 0.5694
Epoch [14/20] | LR: 0.000100 | Train Loss: 1.2390 | Train Acc: 0.9212 | Val Loss: 1.9901 | Val Acc: 0.6157
Epoch [15/20] | LR: 0.000100 | Train Loss: 1.1446 | Train Acc: 0.9421 | Val Loss: 1.9393 | Val Acc: 0.6157
Epoch [16/20] | LR: 0.000100 | Train Loss: 1.1499 | Train Acc: 0.9409 | Val Loss: 1.8941 | Val Acc: 0.6250
Epoch [17/20] | LR: 0.000100 | Train Loss: 1.0987 | Train Acc: 0.9490 | Val Loss: 1.9103 | Val Acc: 0.6435
Epoch [18/20] | LR: 0.000100 | Train Loss: 1.0337 | Train Acc: 0.9664 | Val Loss: 1.8273 | Val Acc: 0.6991
Epoch [19/20] | LR: 0.000100 | Train Loss: 1.0139 | Train Acc: 0.9710 | Val Loss: 1.8906 | Val Acc: 0.6435
Epoch [20/20] | LR: 0.000100 | Train Loss: 0.9960 | Train Acc: 0.9710 | Val Loss: 1.8128 | Val Acc: 0.6574
Best val acc: 0.6990740740740741 at epoch 18
```

Reproducibility:

```python
predictions = []
filenames = []

with torch.no_grad():
    for images, paths in tqdm(test_loader, desc="Predicting on test set"):
        images = images.to(device)
        outputs = model(images)
        _, predicted = torch.max(outputs, dim=1)
        predictions.extend(predicted.cpu().numpy())
        filenames.extend(paths)
print("Predictions generated:", len(predictions))
print("Example filenames:", filenames[:5])
print("Example predictions:", predictions[:5])

submission = pd.DataFrame({
    "ID": filenames,
    "Label": predictions
})

submission["num"] = (
    submission["ID"]
    .str.replace(".jpg", "", regex=False)
    .astype(int)
)

submission = submission.sort_values("num")

submission = submission.drop(columns=["num"])

print(submission.head())
print(submission.tail())

submission_path = "submission5.csv"
submission.to_csv(submission_path, index=False)
```

Example output for submission using the test loop:
Generates 1036 samples. Initially, I had issues with how much lines and samples were supposed to be in the submission csv as the sample submission only had 1000. When submitting my first submission file, an error was returned as there was only 1000 rows when 1036 were needed.

```
dict_keys(['model_state_dict', 'optimizer_state_dict', 'epoch', 'val_acc'])
14
<class '__main__.VIT'>
```

```
Predicting on test set: 100% 33/33 [09:21<00:00, 13.56s/it]
Predictions generated: 1036
Example filenames: ['0.jpg', '1.jpg', '10.jpg', '100.jpg', '1000.jpg']
Example predictions: [np.int64(65), np.int64(43), np.int64(79), np.int64(26), np.int64(31)]
        ID  Label
0    0.jpg     65
1    1.jpg     43
148  2.jpg     38
259  3.jpg     72
370  4.jpg     37
           ID  Label
38   1031.jpg     31
39   1032.jpg      2
40   1033.jpg     12
41   1034.jpg     92
42   1035.jpg     59
Submission saved to: submission4.csv
        ID  Label
0    0.jpg     65
1    1.jpg     43
148  2.jpg     38
259  3.jpg     72
370  4.jpg     37
```

First test set submission:

The first 2 attempted submissions that had errors due to incorrect size and ID.

This was because I implemented my csv file after the sample submission file which had 1000.

submission.csv

Error · 20h ago

```
Submission has 1000 rows but 1036 were expected.
```

Spacing error that I had in my csv file. Fixed using pandas and creating my own submission dataframe.

submission2.csv

Error · 17h ago

```
ID column ID not found in submission
```

Model for both was ViT

submission3.csv

Complete · 17s ago

0.56363

Second test set submission

submission4.csv

Complete · 16s ago

0.55454

Final Accuracy: 0.66363

50	Ethan Phan		0.66363	4